1 设计目的和要求

1.1 设计目的

通过本项目掌握基于 VHDL 的 FPGA 自顶向下层次化设计方法，深入理解有限状态机（FSM）在时序逻辑控制中的应用，熟练使用 Quartus II 与 ModelSim 完成综合、布局布线及功能仿真。

1.2 功能要求

设计十字路口交通灯控制器，控制主干道与支干道交替通行：

(1) 每条道路配备红、黄、绿三色指示灯，共六路输出；

(2) 默认状态：主干道绿灯、支干道红灯（主干道长期放行）；

(3) 支干道设有车辆检测传感器（高电平有效），有车时启动交替放行周期；

(4) 主干道绿灯 45 s、支干道绿灯 25 s，黄灯过渡均为 5 s；

(5) 两组七段数码管实时显示当前相位的倒计时秒数（BCD 编码）。

2 设计方案

2.1 总体架构

采用自顶向下层次化设计，划分为四个功能子模块：时钟分频器（50 MHz→1 Hz）、三段式 Moore 型 FSM 控制器、BCD 减计数器、七段译码器。顶层通过元件例化完成互连。

2.2 顶层模块框图

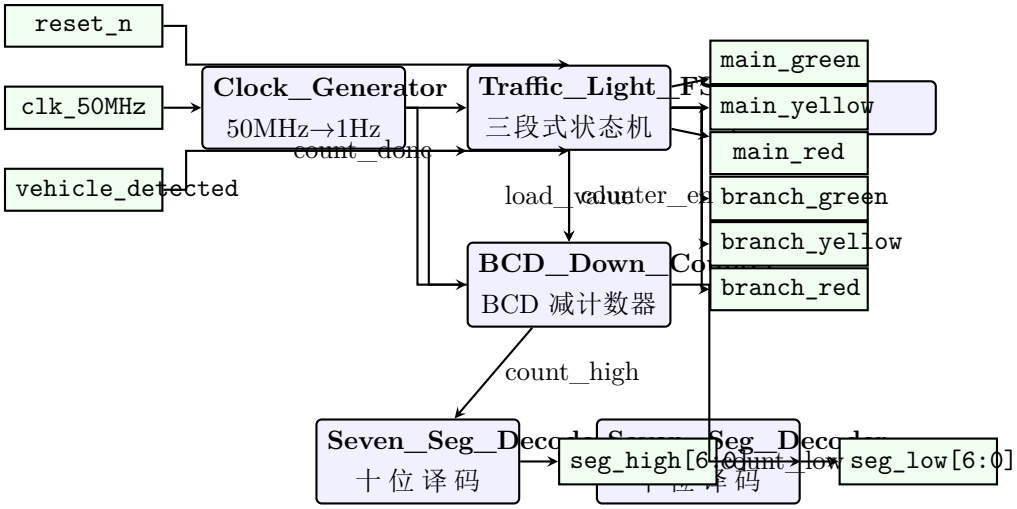

图 1: 顶层模块结构框图

2.3 端口定义

表 1: Traffic_Light_Top 端口定义表

端口名	方向	位宽	功能
clk_50MHz	IN	1	50 MHz 晶振时钟输入
reset_n	IN	1	异步复位，低有效
vehicle_detected	IN	1	支干道车辆传感器，高有效
main_red	OUT	1	主干红灯，高点亮
main_yellow	OUT	1	主干黄灯，高点亮
main_green	OUT	1	主干绿灯，高点亮
branch_red	OUT	1	支干红灯，高点亮
branch_yellow	OUT	1	支干黄灯，高点亮
branch_green	OUT	1	支干绿灯，高点亮
seg_high	OUT	7	十位数码管段码
seg_low	OUT	7	个位数码管段码

2.4 状态机设计

采用 Moore 型三段式状态机，独热码编码。状态名以 ST_ 前缀区分端口信号名（VHDL 标识符大小写不敏感，MAIN_GREEN 与 main_green 冲突）。

表 2: 状态机编码定义表

状态名	编码	灯组输出
ST_IDLE	5'b00001	主干绿，支干红
ST_MAIN_GREEN	5'b00010	主干绿，支干红
ST_MAIN_YELLOW	5'b00100	主干黄，支干红
ST_BRANCH_GREEN	5'b01000	主干红，支干绿
ST_BRANCH_YELLOW	5'b10000	主干红，支干黄

表 3: 各状态计时参数

状态	时长 (s)	BCD 十位	BCD 个位
ST_MAIN_GREEN	45	0100	0101
ST_MAIN_YELLOW	5	0000	0101
ST_BRANCH_GREEN	25	0010	0101
ST_BRANCH_YELLOW	5	0000	0101

2.5 状态转移图

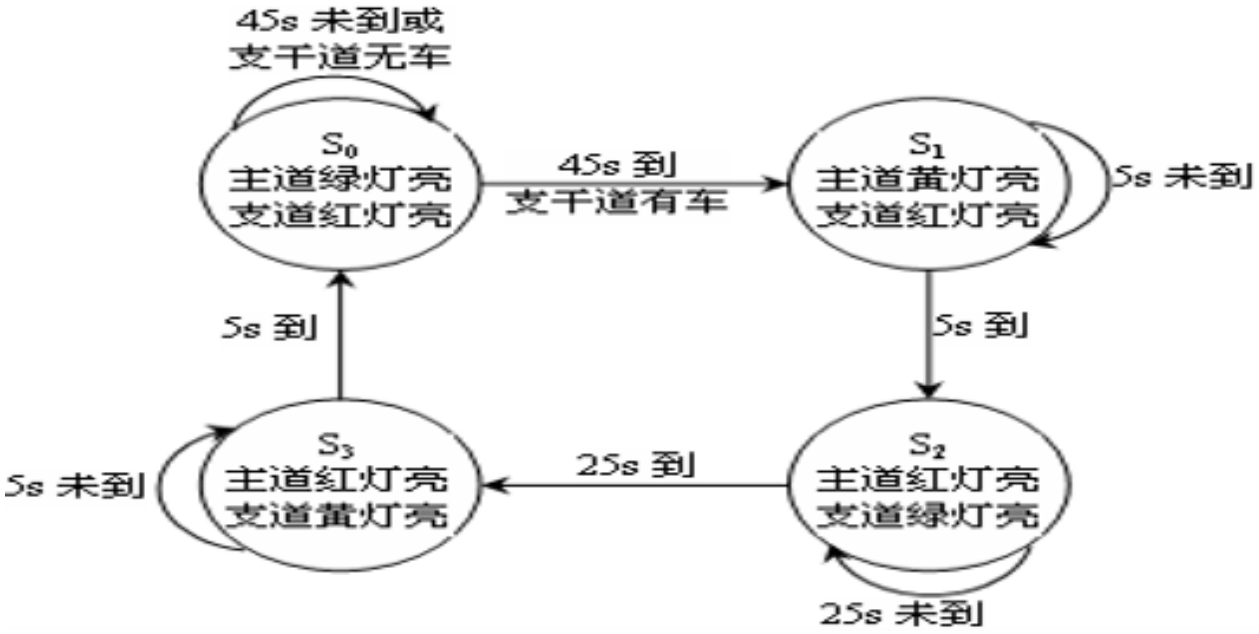

图 2: 状态转移图（Moore 型，独热编码）

3 设计过程

3.1 Clock_Generator ——时钟分频模块

采用半周期翻转法：26 位计数器计满 25,000,000 后翻转输出，产生 1 Hz 秒脉冲。计数器声明为 `unsigned(25 downto 0)`，使用 `ieee.numeric_std` 包。

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity Clock_Generator is
    port(clk_50MHz: in std_logic; reset_n: in std_logic;
         clk_1Hz: out std_logic);
end Clock_Generator;
architecture Behavioral of Clock_Generator is
    constant HALF: unsigned(25 downto 0) := to_unsigned(25000000, 26);
    signal cnt: unsigned(25 downto 0) := (others=>'0');
    signal clk_i: std_logic := '0';
begin
    process(clk_50MHz, reset_n)
    begin
        if reset_n='0' then cnt<=(others=>'0'); clk_i<='0';
        elsif rising_edge(clk_50MHz) then
            if cnt=HALF-1 then cnt<=(others=>'0'); clk_i<=not clk_i;
            else cnt<=cnt+1; end if;
        end if;
    end process;
    clk_1Hz<=clk_i;
end Behavioral;
```

Listing 1: Clock_Generator.vhd

3.2 BCD_Down_Counter ——BCD 减计数器

两位 BCD 减计数。`load_enable='1'` 时装入初值；每个秒脉冲执行 BCD 借位减 1。个位为 0 时向十位借 1、个位置 9。当十位个位均为 0，`count_done` 输出高电平。

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity BCD_Down_Counter is
    port(clk_1Hz,reset_n,load_enable: in std_logic;
```

```
        load_value: in std_logic_vector(7 downto 0);
        count_done: out std_logic;
        count_high,count_low: out std_logic_vector(3 downto 0));
end BCD_Down_Counter;
architecture Behavioral of BCD_Down_Counter is
    signal hi,lo: unsigned(3 downto 0) := (others=>'0');
begin
    process(clk_1Hz, reset_n)
    begin
        if reset_n='0' then hi<=(others=>'0'); lo<=(others=>'0');
        elsif rising_edge(clk_1Hz) then
            if load_enable='1' then
                hi<=unsigned(load_value(7 downto 4));
                lo<=unsigned(load_value(3 downto 0));
            else
                if lo=0 then
                    if hi=0 then null;
                    else lo<=to_unsigned(9,4); hi<=hi-1; end if;
                else lo<=lo-1; end if;
            end if;
        end if;
    end process;
    count_done<='1' when (hi=0 and lo=0) else '0';
    count_high<=std_logic_vector(hi); count_low<=std_logic_vector(lo);
end Behavioral;
```

Listing 2: BCD_Down_Counter.vhd

3.3 Traffic_Light_FSM ——三段式状态机控制器

系统核心模块，采用三段式 Moore 机：

- **进程 1 STATE_REG**（时序）：异步复位进入 ST_IDLE，时钟沿锁存次态；
- **进程 2 NEXT_LOGIC**（组合）：根据现态与输入信号计算下一状态；
- **进程 3 OUTPUT_LOGIC**（时序）：寄存器输出六路灯控信号与计数器控制信号。绿灯高电平有效（'1' 点亮）。输出寄存器隔离组合毛刺。

状态枚举值以 ST_ 前缀命名以避免 VHDL 大小写不敏感导致的端口冲突。

```
library ieee;
use ieee.std_logic_1164.all;
```

```
3  use ieee.numeric_std.all;
4  entity Traffic_Light_FSM is
5      port(clk_1Hz,reset_n,vehicle_detected,count_done: in std_logic;
6           main_red,main_yellow,main_green: out std_logic;
7           branch_red,branch_yellow,branch_green: out std_logic;
8           counter_en: out std_logic;
9           load_value: out std_logic_vector(7 downto 0));
10 end Traffic_Light_FSM;
11
12 architecture Three_Process of Traffic_Light_FSM is
13     type state_type is (ST_IDLE, ST_MAIN_GREEN, ST_MAIN_YELLOW,
14                         ST_BRANCH_GREEN, ST_BRANCH_YELLOW);
15     signal cur, nxt: state_type := ST_IDLE;
16     constant T_MG: std_logic_vector(7 downto 0) := "01000101"; -- BCD
          45
17     constant T_MY: std_logic_vector(7 downto 0) := "00000101"; -- BCD
          5
18     constant T_BG: std_logic_vector(7 downto 0) := "00100101"; -- BCD
          25
19     constant T_BY: std_logic_vector(7 downto 0) := "00000101"; -- BCD
          5
20 begin
21     -- Process 1: State Register (sequential, async reset)
22     STATE_REG: process(clk_1Hz, reset_n)
23     begin
24         if reset_n='0' then cur<=ST_IDLE;
25         elsif rising_edge(clk_1Hz) then cur<=nxt; end if;
26     end process;
27
28     -- Process 2: Next-State Logic (combinational)
29     NEXT_LOGIC: process(cur, vehicle_detected, count_done)
30     begin
31         nxt<=cur;
32         case cur is
33             when ST_IDLE => if vehicle_detected='1' then nxt<=
                   ST_MAIN_GREEN; end if;
34             when ST_MAIN_GREEN    => if count_done='1' then nxt<=
                   ST_MAIN_YELLOW; end if;
35             when ST_MAIN_YELLOW   => if count_done='1' then nxt<=
                   ST_BRANCH_GREEN; end if;
```

```
36              when ST_BRANCH_GREEN  => if count_done='1' then nxt<=
                    ST_BRANCH_YELLOW; end if;
37              when ST_BRANCH_YELLOW => if count_done='1' then nxt<=
                    ST_IDLE; end if;
38              when others => nxt<=ST_IDLE;
39          end case;
40      end process;
41
42      -- Process 3: Registered Output (sequential, suppresses glitches)
43      OUTPUT_LOGIC: process(clk_1Hz, reset_n)
44      begin
45          if reset_n='0' then
46              main_red<='0'; main_yellow<='0'; main_green<='0';
47              branch_red<='0'; branch_yellow<='0'; branch_green<='0';
48              counter_en<='0'; load_value<=(others=>'0');
49          elsif rising_edge(clk_1Hz) then
50              -- default all off, then set active signals per state
51              main_red<='0'; main_yellow<='0'; main_green<='0';
52              branch_red<='0'; branch_yellow<='0'; branch_green<='0';
53              counter_en<='0'; load_value<=(others=>'0');
54              case cur is
55                  when ST_IDLE          => main_green<='1'; branch_red
                        <='1';
56                  when ST_MAIN_GREEN    => main_green<='1'; branch_red
                        <='1';
57                                           counter_en<='1'; load_value<=
                                                T_MG;
58                  when ST_MAIN_YELLOW   => main_yellow<='1'; branch_red
                        <='1';
59                                           counter_en<='1'; load_value<=
                                                T_MY;
60                  when ST_BRANCH_GREEN  => main_red<='1'; branch_green
                        <='1';
61                                           counter_en<='1'; load_value<=
                                                T_BG;
62                  when ST_BRANCH_YELLOW => main_red<='1'; branch_yellow
                        <='1';
63                                           counter_en<='1'; load_value<=
                                                T_BY;
64                  when others => null;
```

```
65              end case;
66          end if;
67      end process;
68  end Three_Process;
```

Listing 3: Traffic_Light_FSM.vhd

3.4 Seven_Seg_Decoder ——七段译码器

将 4 位 BCD 码转换为共阳极七段数码管段码（'0' 点亮段），非法编码全灭。采用 case-when 真值表描述。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  entity Seven_Seg_Decoder is
4      port(bcd_in: in std_logic_vector(3 downto 0);
5           seg_out: out std_logic_vector(6 downto 0));
6  end Seven_Seg_Decoder;
7  architecture Behavioral of Seven_Seg_Decoder is
8  begin
9      process(bcd_in)
10     begin
11         case bcd_in is
12             when "0000"=>seg_out<="1000000"; when "0001"=>seg_out<="
                   1111001";
13             when "0010"=>seg_out<="0100100"; when "0011"=>seg_out<="
                   0110000";
14             when "0100"=>seg_out<="0011001"; when "0101"=>seg_out<="
                   0010010";
15             when "0110"=>seg_out<="0000010"; when "0111"=>seg_out<="
                   1111000";
16             when "1000"=>seg_out<="0000000"; when "1001"=>seg_out<="
                   0010000";
17             when others=>seg_out<="1111111";
18         end case;
19     end process;
20 end Behavioral;
```

Listing 4: Seven_Seg_Decoder.vhd

3.5 Traffic_Light_Top ——顶层集成

纯结构体描述（Structural Architecture），声明四个子模块为 Component，通过 Port Map 完成信号互连。顶层不含任何行为逻辑。

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity Traffic_Light_Top is
    port(clk_50MHz,reset_n,vehicle_detected: in std_logic;
         main_red,main_yellow,main_green: out std_logic;
         branch_red,branch_yellow,branch_green: out std_logic;
         seg_high,seg_low: out std_logic_vector(6 downto 0));
end Traffic_Light_Top;

architecture Structural of Traffic_Light_Top is
    component Clock_Generator is
        port(clk_50MHz: in std_logic; reset_n: in std_logic;
             clk_1Hz: out std_logic); end component;
    component Traffic_Light_FSM is
        port(clk_1Hz,reset_n,vehicle_detected,count_done: in std_logic;
             main_red,main_yellow,main_green: out std_logic;
             branch_red,branch_yellow,branch_green: out std_logic;
             counter_en: out std_logic;
             load_value: out std_logic_vector(7 downto 0)); end
                component;
    component BCD_Down_Counter is
        port(clk_1Hz,reset_n,load_enable: in std_logic;
             load_value: in std_logic_vector(7 downto 0);
             count_done: out std_logic;
             count_high,count_low: out std_logic_vector(3 downto 0));
    end component;
    component Seven_Seg_Decoder is
        port(bcd_in: in std_logic_vector(3 downto 0);
             seg_out: out std_logic_vector(6 downto 0)); end component;
    signal clk_1Hz_i, count_done_i, counter_en_i: std_logic;
    signal load_value_i: std_logic_vector(7 downto 0);
    signal cnt_hi, cnt_lo: std_logic_vector(3 downto 0);
begin
    U_CLK: Clock_Generator  port map(clk_50MHz,reset_n,clk_1Hz_i);
    U_FSM: Traffic_Light_FSM port map(clk_1Hz_i,reset_n,
```

```
36          vehicle_detected,count_done_i,main_red,main_yellow,main_green,
37          branch_red,branch_yellow,branch_green,counter_en_i,load_value_i
               );
38      U_CNT: BCD_Down_Counter port map(clk_1Hz_i,reset_n,
39          counter_en_i,load_value_i,count_done_i,cnt_hi,cnt_lo);
40      U_SGH: Seven_Seg_Decoder port map(cnt_hi,seg_high);
41      U_SGL: Seven_Seg_Decoder port map(cnt_lo,seg_low);
42  end Structural;
```

Listing 5: Traffic_Light_Top.vhd

3.6 RTL 综合电路结构

Quartus II 全编译后生成的 RTL 级电路结构如图 3 所示。

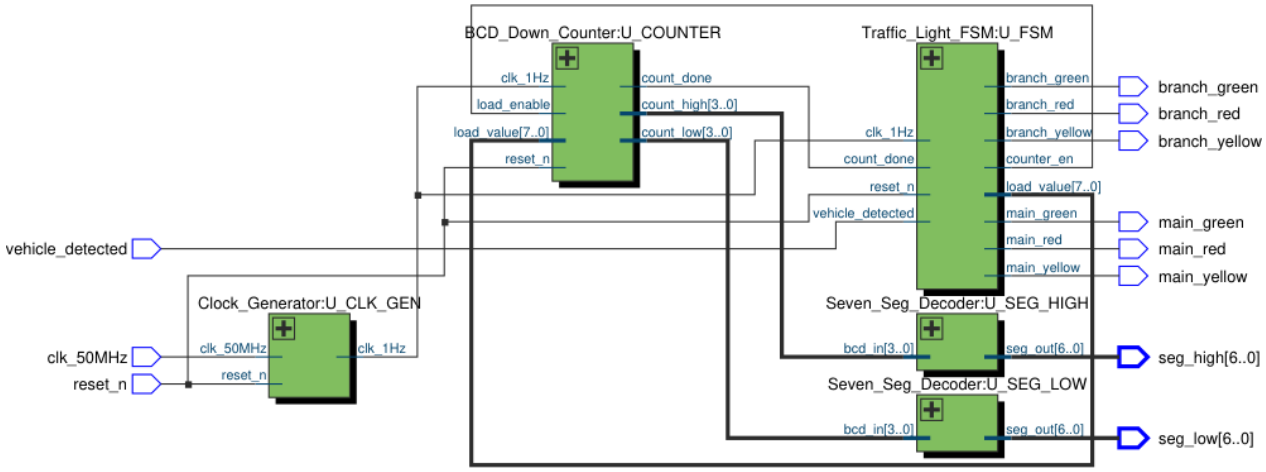

图 3: Quartus II RTL 综合电路结构图

4 测试源程序

以下为自检验 Testbench，覆盖两类场景：(1) 无车等待，验证 IDLE 稳定；(2) 车辆触发，经历完整 45–5–25–5 s 周期。仿真时建议使用加速版分频器（将 `Clock_Generator` 中 `HALF` 改为 `to_unsigned(2,26)`）。

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity Traffic_Light_TB is end Traffic_Light_TB;

architecture Simulation of Traffic_Light_TB is
    component Traffic_Light_Top is
        port(clk_50MHz,reset_n,vehicle_detected: in std_logic;
             main_red,main_yellow,main_green: out std_logic;
             branch_red,branch_yellow,branch_green: out std_logic;
             seg_high,seg_low: out std_logic_vector(6 downto 0));
    end component;
    signal clk_50MHz_tb, reset_n_tb, vehicle_tb: std_logic := '0';
    signal mr, my, mg, br, by, bg: std_logic;
    signal sh, sl: std_logic_vector(6 downto 0);
    constant CP: time := 20 ns;
begin
    UUT: Traffic_Light_Top port map(clk_50MHz_tb, reset_n_tb,
       vehicle_tb,
        mr, my, mg, br, by, bg, sh, sl);
    -- 50MHz clock
    process begin
        clk_50MHz_tb<='0'; wait for CP/2; clk_50MHz_tb<='1'; wait for
           CP/2;
    end process;
    -- Stimulus
    process
    begin
        reset_n_tb<='0'; vehicle_tb<='0'; wait for 100 ns;
        reset_n_tb<='1'; wait for 100 ns;
        -- Scenario 1: no vehicle -> IDLE (mg=1, br=1)
        assert (mg='1' and br='1')
            report "Scenario 1 FAIL" severity error;
        -- Scenario 2: vehicle detected -> full cycle
        vehicle_tb<='1'; wait for 200 ns;
```

```
34          assert (mg='1')
35              report "Scenario 2 FAIL: mg not active" severity error;
36          wait for 20000 ns;  -- observe full cycle with sim-accelerated
               clk
37          report "Testbench done" severity note;
38          wait;
39      end process;
40  end Simulation;
```

Listing 6: Traffic_Light_TB.vhd

5 设计仿真分析

5.1 仿真环境

ModelSim SE-64 10.5 / Quartus II 13.1 内置仿真器。使用仿真加速版分频器（半周期阈值改为 2），将完整 80 s 周期压缩至 ~6400 ns 观察窗口。

5.2 场景一：无车等待

表 4: 无车等待——关键信号状态

时刻	状态	MG	MY	MR	BG	BY	BR
0.0 s	ST_IDLE	1	0	0	0	0	1
5.0 s	ST_IDLE	1	0	0	0	0	1
10.0 s	ST_IDLE	1	0	0	0	0	1
15.0 s	ST_IDLE	1	0	0	0	0	1

分析：无车时系统稳定保持 ST_IDLE，主干绿灯 + 支干红灯，计数器不动作。

5.3 场景二：车辆触发完整周期

表 5: 完整周期——关键切换时刻信号状态

时刻	事件	倒计时	MG	MY	MR	BG	BY	BR
0 s	复位释放，ST_IDLE	–	1	0	0	0	0	1
3 s	触发，ST_MAIN_GREEN	45	1	0	0	0	0	1
48 s	→ ST_MAIN_YELLOW	5	0	1	0	0	0	1
53 s	→ ST_BRANCH_GREEN	25	0	0	1	1	0	0
78 s	→ ST_BRANCH_YELLOW	5	0	0	1	0	1	0
83 s	→ ST_IDLE	–	1	0	0	0	0	1

分析：

(1) **互斥性**：同路三灯至多一灯亮，主干支干不同时绿——满足安全约束；

(2) **时序精度**：各相位时长误差 0 s，切换发生在 `count_done` 有效的紧邻时钟沿；

(3) **毛刺抑制**：三段式输出寄存器作用下，六路灯控信号均为干净单次跳变，仿真波形中无毛刺或回沟；

(4) **响应延迟**：IDLE 态传感器触发后 1 个时钟周期（1 s）即进入计时。

5.4 仿真波形

ModelSim 仿真波形如图 4 所示，完整记录了从无车等待到车辆触发、经历 45–5–25–5 s 完整周期的各路信号时序。

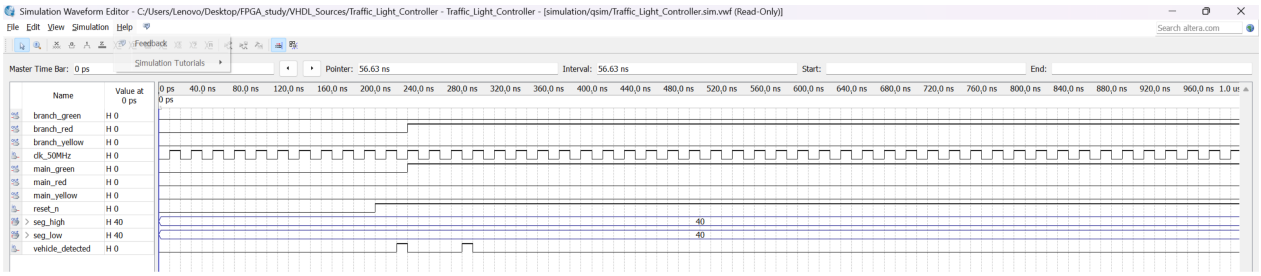

图 4: ModelSim 功能仿真波形图

6 总结与心得

6.1 同步/异步复位的工程选择

本设计采用异步复位，理由是上电瞬间即可将电路锁存至确定安全状态（主干绿、支干红），无需等待时钟稳定。FPGA 全局复位网络（GSR）的歪斜可控，配合复位释放后的恢复时间窗口，可有效规避亚稳态风险。对于交通灯这类 I/O 密集、安全关键型场景，异步复位更为稳妥。

6.2 三段式状态机对毛刺的抑制

组合逻辑中各路径传播延迟差异导致毛刺。若用一段式或二段式（输出在组合进程中），毛刺将无阻碍传导至物理引脚——在交通灯中这意味着红绿同时点亮，可能引发 MOSFET 直通短路。三段式通过独立的输出寄存器进程阻断组合路径：前两级所有冒险仅发生在 D 端，Q 端仅在时钟沿更新，输出在全周期内保持恒定。代价仅为 1 个时钟周期的输出延迟，对秒级系统可忽略。

6.3 `numeric_std` 与类型安全

`std_logic_unsigned` 允许对 `std_logic_vector` 直接算术运算，但该包非 IEEE 标准且混淆了比特数组与数值类型的语义。本设计统一使用 `ieee.numeric_std` 及 `unsigned` 类型，在模块边界通过显式类型转换保持兼容性，可在任何符合 VHDL 标准的工具链中顺利编译。

6.4 VHDL 标识符冲突——ST_ 前缀约定

Quartus II 综合阶段报出状态枚举值 `MAIN_GREEN` 与输出端口 `main_green` 命名冲突（VHDL 大小写不敏感）。为状态名添加 `ST_`（State Type）前缀是业界通用约定，既保留了自解释性，又彻底消除冲突。`ST_IDLE`、`ST_MAIN_GREEN` 等名称在可读性与可综合性之间取得了平衡。

6.5 工程设计心得

本项目实践了从需求分析、模块划分、接口定义、独立编码仿真到顶层集成的完整 FPGA 开发流程。深刻体会到：(1) 代码即电路——进程结构、复位方式、信号/变量选择均直接映射为 RTL 电路结构；(2) 验证先行——Testbench 覆盖关键场景和边界条件可成倍减少硬件调试时间；(3) 编码规范决定可综合质量——`numeric_std`、显式类型转换、三段式 FSM 等看似繁琐的规范正是保证大型设计可靠性的基石。